# Cahier des charges — Module 114 : Codification / Chiffrement

## Projet : Stéganographie

*Créé le 2025-09-29 — révisé le 2026-08-21*

> **Consigne :** l'utilisation de Copilot ou de tout autre outil d'IA générative est interdite pour ce projet.

## Objectif du projet

Ajouter au programme Couteau Suisse un utilitaire qui permet de cacher (encoder) un message texte en Morse à l'intérieur d'un texte « porteur » saisi par l'utilisateur, puis de le retrouver (décoder) ultérieurement.

Les fichiers `.txt` en UTF-8 sont utilisés pour stocker le résultat de la stéganographie ; ils servent également d'entrée pour l'option de décodage.

L'effet recherché : le texte stéganographié contenu dans le fichier `.txt` a l'air normal, mais contient des marqueurs invisibles qui codent le Morse.

## Objectifs pédagogiques

Ce projet permet de :

- Comprendre un principe de stéganographie (cacher de l'information dans un support qui semble anodin)
- Manipuler l'encodage Unicode et des caractères invisibles
- Concevoir deux fonctions symétriques (`Encode` / `Decode`) qui doivent rester cohérentes entre elles
- Combiner plusieurs notions déjà vues (dictionnaire, boucles, conditions, fonctions) sur un cas plus complexe
- Lire et écrire des fichiers texte en UTF-8
- Renforcer la pratique de la modélisation et de l'utilisation de critères d'acceptation pour valider un travail

## Exemple d'exécution console

```
=== Couteau Suisse – Utilitaires ===
1. Convertir du texte en code Morse
2. Convertir des nombres entre différentes bases (Décimal <> Binaire <> Octal)
3. Stéganographie
Veuillez entrer votre choix : 3

=== Stéganographie ===
1. Encoder (Cacher un message)
2. Décoder (Extraire un message caché)
Veuillez entrer votre choix : 1
Texte porteur (ce que l'on verra) :
Les informaticiens aiment les défis.
Message secret à cacher (A-Z et espace) :
SOS
Le texte avec stéganographie a été sauvegardé dans le fichier stegano.txt
```

## Fonctionnalités attendues

| Fonction | Description |
|---|---|
| Menu « Stéganographie » | Ajoute une entrée dans le menu principal avec deux options : Encoder et Décoder |
| Encodage | Permet de saisir un texte porteur et un message secret, puis produit un texte stéganographié visuellement identique mais contenant le message en Morse caché avec des caractères invisibles |
| Décodage | Permet de coller un texte stéganographié et de récupérer le message secret caché en Morse |
| Validation des entrées | Contrôle que le message secret est bien composé de lettres A–Z et d'espaces (converties en majuscules), et que le texte porteur n'est pas vide |
| Gestion des erreurs | Signale les problèmes (texte porteur trop court, caractères non valides, texte sans marqueurs invisibles) de manière claire et pédagogique |
| Affichage copiable | Le texte stéganographié produit doit pouvoir être copié/collé tout en gardant les marqueurs invisibles actifs |
| Intégration | S'intègre au programme Couteau Suisse existant sans casser les autres fonctionnalités (Morse simple, bases numériques) |

## 1. Marqueurs invisibles

Encodage « marqueurs invisibles » (Unicode) : on insère des caractères de largeur nulle après des caractères du texte porteur pour coder le Morse.

| Élément à encoder | Caractère en C# | Définition du caractère |
|---|---|---|
| Le point | `\u200B` | ZERO WIDTH SPACE |
| Le tiret | `\u200C` | ZERO WIDTH NON-JOINER |
| Séparateur de lettre | `\u200D` | ZERO WIDTH JOINER |
| Séparateur de mot | `\u2060` | WORD JOINER |

## 2. Concepts informatiques

- Variables (chaînes de caractères, entiers)
- Tableau associatif / dictionnaire (table de correspondance lettre ↔ Morse)
- Boucles (`foreach`, `for`) pour parcourir le texte porteur et le message secret
- Conditions (`if`, `else`, `switch`) pour déterminer quel caractère invisible insérer ou lire
- Fonctions simples et spécialisées :
  - `Encode(string coverText, string secret)`
  - `Decode(string stegoText)`
  - `ConvertToMorse(string text)`
  - `ConvertFromMorse(string morse)`
- Concaténation et manipulation de chaînes
- Encodage Unicode (caractères invisibles `\u200B`, `\u200C`, etc.)

## 3. Modélisation

Avant d'écrire la moindre ligne de code, l'élève doit modéliser sa fonction principale. **Trois notations sont acceptées** — au choix de l'élève :

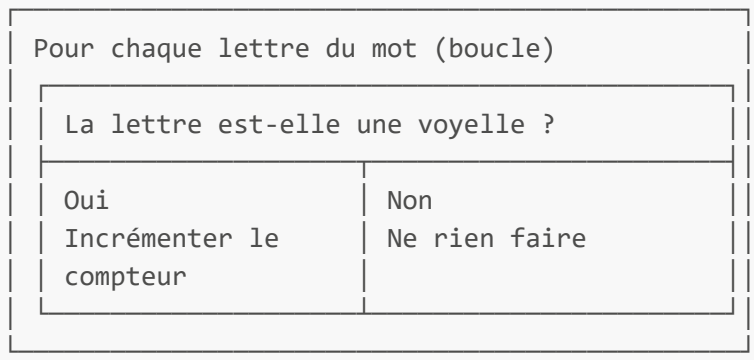
- **Structogramme (Nassi-Shneiderman)** : blocs imbriqués sans flèches, très lisible pour représenter séquence / condition / boucle.
- **Pseudocode structuré** : texte semi-formel, rapide à écrire, se transpose presque ligne à ligne en C#.
- **Organigramme (flowchart)** : boîtes reliées par des flèches, plus visuel.

La modélisation préalable est particulièrement importante et doit couvrir **au minimum** la fonction `Encode` (parcours du texte porteur, insertion des marqueurs invisibles selon le Morse du message secret) qu'il faut définir et modéliser avant de coder.

**Exemple — les 3 notations pour la même logique**

*Exemple générique (comptage de voyelles), sans lien avec la logique du projet, pour ne pas donner la solution.*

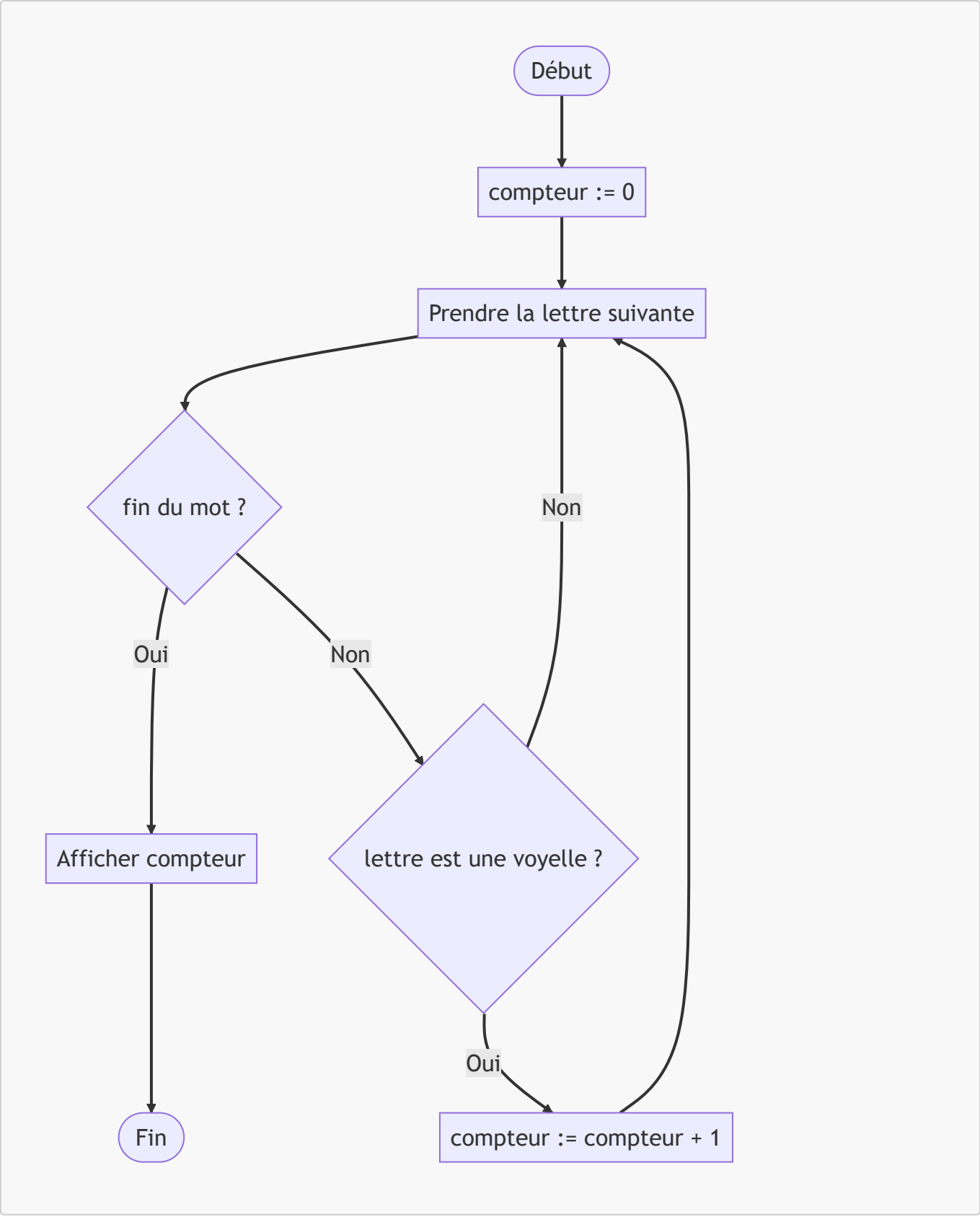
**Structogramme :**




**Pseudocode :**

```
FONCTION CompterVoyelles(mot)
    compteur ← 0
    POUR CHAQUE lettre DANS mot
        SI lettre EST DANS ("A","E","I","O","U","Y") ALORS
            compteur ← compteur + 1
        FIN SI
    FIN POUR
    RETOURNER compteur
FIN FONCTION
```

**Organigramme (flowchart) :**




## 4. Structure du programme

1. Accueil et explication du programme
2. Choix de l'utilitaire
3. Affichage et explication du sous-programme
4. Modélisation (structogramme, pseudocode ou organigramme)

5. Conversion
6. Affichage/Stockage du résultat

## 5. Validation — critères d'acceptation

Avant de considérer le projet terminé, on vérifie que chacun des points suivants est rempli (première approche des tests d'acceptance) :

- ☐ Un modèle (structogramme, pseudocode ou organigramme) de la fonction `Encode` a été produit avant le codage
- ☐ Le menu principal propose l'option « Stéganographie » avec Encoder / Décoder
- ☐ Un message secret encodé puis décodé redonne exactement le même message (« round-trip »)
- ☐ Le texte porteur reste visuellement identique après encodage (aucun caractère visible ajouté)
- ☐ Un message secret contenant des caractères non A-Z/espace est rejeté ou corrigé (mise en majuscules)
- ☐ Un texte porteur trop court pour cacher le message est signalé clairement
- ☐ Un texte sans marqueurs invisibles est détecté comme tel lors du décodage (pas de plantage)

## Volume horaire

Volume indicatif pour ce projet : **~10 périodes** (environ 3 semaines, à raison de 3 périodes/semaine).

*Estimation ajustable selon le rythme de l'élève et en accord avec l'enseignant.*

## Outils et environnement

- **Langage :** C# console (application console, pas d'interface graphique)
- **IDE :** Visual Studio Code ou Visual Studio Community

## Évolutions

- Ajout d'un mode DEBUG qui permet de visualiser les caractères cachés (ex. en remplaçant chaque marqueur invisible par un symbole visible : `.` pour `\u200B`, `-` pour `\u200C`, etc.)
- Paramètres de ligne de commande : passer directement les chemins des fichiers d'entrée/sortie en argument (ex. `Stegano.exe --encode porteur.txt --out stegano.txt`)
- Colorisation de la console : en mode DEBUG, afficher les marqueurs invisibles détectés dans une couleur différente du texte porteur
- Chiffrement supplémentaire du message secret (ex. XOR simple avec une clé) avant l'encodage stéganographique, pour ajouter une couche de sécurité
- Tests unitaires sur les fonctions `Encode`/`Decode` (ex. avec xUnit)

*Ces évolutions sont facultatives, à réaliser une fois les fonctionnalités de base validées — elles ne sont pas comptées dans le volume horaire indiqué ci-dessus.*